

Module Documentation: File Import and Validation Subsystem

1. Overview

The `XmlValidator.ts` module provides validation logic for XML-based file uploads as described in [SAS 7.1](#) and in the [BaSyx WebUI issue](#).

It validates the following file formats in particular:

- KBL
- VEC
- generic XML
- IODD XML

The validation checks file extensions, XML well-formedness, XML headers, format-dependent root elements, and, for IODD files, additional mandatory core structures.

2. Purpose

The module is intended to validate uploaded XML-based files before further processing.

Its objective is to detect malformed or technically unsuitable files at an early stage and return a comprehensible error message to the calling components.

Core functions:

- Validation of uploaded files
- Routing based on file extension
- XML header validation
- XML well-formedness validation
- Validation of allowed root elements for KBL and VEC
- Detection of IODD documents
- Validation of mandatory IODD core elements
- Return of an error message for UI components

3. Responsibilities

The module is responsible for:

Area	Description
Upload validation	Validation of a provided <code>File</code> object
Format routing	Selection of the appropriate validation logic based on the file extension
XML base validation	Validation of header, syntax, and parser errors
KBL validation	Validation of allowed root elements for <code>.kbl</code> files
VEC validation	Validation of allowed root elements for <code>.vec</code> files
XML/IODD validation	Acceptance of generic XML files and additional validation for IODD
IODD mandatory-element validation	Verification that mandatory IODD structures are present
Error messages	Provision of specific error messages for calling components

Mandatory IODD Elements

For detected IODD documents, the validator checks whether the following mandatory elements or structures are present:

Element / Structure	Requirement
<code>DeviceIdentity</code>	Must be present
<code>DeviceFunction</code>	Must be present
<code>DeviceIdentity/DeviceName</code> or <code>DeviceVariant/ProductName</code>	At least one of these naming structures must be present
<code>ExternalTextCollection/PrimaryLanguage</code>	Must be present if <code>ExternalTextCollection</code> exists

The module is not primarily responsible for:

Non-responsibility	Reason
Complete technical KBL/VEC validation	It only checks basic structural and root-element rules
Processing or extraction of XML data	The module validates only; it does not transform data
File persistence	Storage is handled outside the validator
UI rendering of errors	Error texts are returned but not displayed by the module itself
Collection of all validation errors	Validation stops at the first detected error

4. Public API

```
uploadHandler(fileInput: File | null): Promise<string>
```

Central entry point for validating uploaded files.

Parameters

Name	Type	Required	Description
<code>fileInput</code>	<code>File</code> <code>null</code>	yes	File object to be validated

Return Value

Return value	Meaning
<code>""</code>	Validation successful or no validation required
<code>string</code>	Error message if validation failed

Behavior

The method determines which specific validation function is used based on the file extension.

File extension	Validation function
<code>.vec</code>	<code>validateVecFile()</code>
<code>.kbl</code>	<code>validateKBLFile()</code>
<code>.xml</code>	<code>validateXmlFile()</code>
other	no specific validation; returns an empty error string

Example

```
const validationError = await uploadHandler(file)

if (validationError !== '') {
  console.error('Validation failed:', validationError)
}
```

5. Core Validation Functions

```
5.1 validateWellFormedXML(f: File): Promise<ValidatorResult>
```

Base step for all XML-based validations.

Purpose

Checks whether a file can be read and parsed as XML.

Checks

Check	Description
XML header	The file must contain an XML declaration with <code><?xml</code>
XML syntax	The file is parsed with <code>DOMParser</code>
Parser errors	<code>parsererror</code> elements are detected

Return Value

Return value	Meaning
<code>{ ok: true, doc: Document }</code>	XML is well-formed
<code>{ ok: false, doc: null }</code>	XML is malformed or could not be parsed

Possible Error Messages

Error message	Cause
<code>XML header is missing.</code>	No XML declaration is present
<code>XML is not well-formed.</code>	Parser error detected
<code>Error while parsing the XML file.</code>	Exception during parsing

```
5.2 validateVecFile(f: File): Promise<boolean>
```

Validates `.vec` files.

Checks

Check	Description
File extension	The file must end with <code>.vec</code>
XML well-formedness	Checked via <code>validateWellFormedXML()</code>
Root element	The root element must match an allowed VEC root

Allowed Root Elements

The check is case-insensitive.

Root element
<code>VecContent</code>
<code>VecContentV2</code>
<code>VecContent_Base</code>

Possible Error Messages

Error message	Cause
<code>Only .vec files are allowed.</code>	The file has an invalid extension
<code>No root element found.</code>	No root element was found
<code>`\${localName}` is not a valid root element for .vec files.</code>	The root element is not allowed for VEC

```
5.3 validateKBLFile(f: File): Promise<boolean>
```

Validates `.kbl` files.

Checks

Check	Description
File extension	The file must end with <code>.kbl</code>
XML well-formedness	Checked via <code>validateWellFormedXML()</code>
Root element	The root element must match an allowed KBL root

Allowed Root Elements

The check is case-insensitive.

Root element
<code>KBL_container</code>
<code>KBLContainer</code>
<code>KBL_Container</code>
<code>KBLContainer_Old</code>

Possible Error Messages

Error message	Cause
<code>Only .kbl files are allowed.</code>	The file has an invalid extension
<code>No root element found.</code>	No root element was found
<code>`\${localName}` is not a valid root element for .kbl files.</code>	The root element is not allowed for KBL

```
5.4 validateXmlFile(f: File): Promise<boolean>
```

Validates `.xml` files.

Flow

```
1. Check XML well-formedness.
2. Detect the IODD namespace.
3. If IODD is detected:
   3.1 Validate the IODD core structure.
4. If no IODD is detected:
   4.1 Accept the file as generic XML.
```

IODD Namespace

```
http://www.iso-link.com/IODD/2010/10
```

```
5.5 validateIoddCore(doc: Document): boolean
```

Validates central structures of an IODD document.

Mandatory Structures

Element / Structure	Description
<code>DeviceIdentity</code>	Container for device metadata
<code>DeviceFunction</code>	Functional specification
<code>DeviceIdentity/DeviceName</code> or <code>DeviceVariant/ProductName</code>	Device name or fallback product name
<code>ExternalTextCollection/PrimaryLanguage</code>	Mandatory if <code>ExternalTextCollection</code> exists

Possible Error Messages

Error message	Cause
<code>IODD element 'DeviceIdentity' is missing.</code>	<code>DeviceIdentity</code> is missing
<code>IODD element 'DeviceFunction' is missing.</code>	<code>DeviceFunction</code> is missing
<code>IODD element 'DeviceIdentity/DeviceName' is missing, and fallback 'DeviceVariant/ProductName' is also missing.</code>	No device name and no fallback product name are present
<code>IODD element 'ExternalTextCollection' exists, but 'PrimaryLanguage' is missing.</code>	<code>ExternalTextCollection</code> exists without <code>PrimaryLanguage</code>

6. Helper Functions

6.1 XML Element Queries

Function	Purpose
<code>hasElement(doc: Document, tagName: string): boolean</code>	Checks whether an element exists anywhere in the document
<code>getFirstElement(doc: Document, tagName: string): Element null</code>	Returns the first matching element
<code>hasDirectChildElement(parent: Element null, tagName: string): boolean</code>	Checks whether a direct child element exists

6.2 IODD Detection

Function	Purpose
<code>isIoddDoc(doc: Document): boolean</code>	Checks whether the document uses the IODD namespace

6.3 IODD Detail Checks

Function	Purpose
<code>hasDeviceNameOrVariantProductName(doc: Document): boolean</code>	Checks whether a device name or fallback product name is present
<code>hasPrimaryLanguageInExternalTextCollection(doc: Document): boolean</code>	Checks whether an existing <code>ExternalTextCollection</code> contains <code>PrimaryLanguage</code>

7. Module State

The module uses module-level variables to manage the current validation state.

```
let file: File | null = null
let errorMessage: string = ''
let resultMessage: string = ''
```

Variable	Meaning
<code>file</code>	File currently being validated
<code>errorMessage</code>	Most recent error message
<code>resultMessage</code>	Most recent success message or diagnostic information

Important:

- `uploadHandler()` returns `errorMessage`.
- `resultMessage` is only relevant internally or for diagnostics.
- Because of the module-level state, parallel validations should be handled with care.

8. Data and Control Flow

```
File | null
├─ uploadHandler()
│   └─ Check file extension
│       └─ Format-dependent validation function
│           ├─ .vec → validateVecFile()
│           ├─ .kbl → validateKBLFile()
│           └─ .xml → validateXmlFile()
│               └─ other → no specific validation
├─ validateWellFormedXML()
│   └─ DOMParser
│       └─ Root-element or IODD-structure validation
├─ errorMessage
└─ Return to component
```

9. Use in Components

9.1 Use in `UploadAASKblVec.vue`

```
const validationError = await uploadHandler(file)

if (validationError && validationError.trim() !== '') {
  // Show the error to the user
  return
}
```

9.2 Use in `FileForm.vue`

```
const errorMsg = await uploadHandler(fileElement.value)

if (errorMsg !== '') {
  errors.value.set('fileInput', errorMsg)
  return
}
```

10. Test Coverage

The tests are located in:

```
tests/utlils/XMLValidator.test.ts
```

The tests cover the following areas:

Area	Examples
KBL validation	missing header, invalid root element, well-formed KBL, malformed KBL
VEC validation	valid VEC, invalid VEC root, missing version tag
Namespace handling	VEC inside KBL, KBL inside VEC
Malformed XML	unclosed tags, invalid declarations
Cross-format errors	VEC content in <code>.kbl</code> , KBL content in <code>.vec</code>

11. Technical Notes

DOMParser

The module uses `DOMParser` to parse XML.

MIME type used by the parser:

```
text/xml
```

Important:

- `DOMParser` is browser-based.
- Parser errors are detected via `parsererror` elements.

MIME Type Detection and Automatic shortID Population

Additional MIME types used for the supported KBL and VEC files are:

```
application/kbl+xml
application/vec+xml
```

These MIME types are defined in:

```
src\components\EditorComponents\InputTypes\FileInput.vue
```

at lines [121](#) and [122](#).

MIME type detection is implemented in:

```
src\composables\AAS\SubmodelElements\File.ts
```

at lines [227](#) to [229](#).

Case-insensitive Root Matching

Root element names are normalized before comparison.

This allows spelling variants of the permitted root elements to be accepted.

Namespace-aware Queries

Element queries are namespace-aware.

For this purpose, queries such as the following are used:

```
getElementsByTagNameNS('*', name)
```

This makes it possible to validate XML documents that use namespaces.

Early Exit

Validation stops at the first error.

The module does not attempt to collect all errors in a document.

Automatic Population of ShortID

This module is also responsible for automatically populating the `shortID`Field based on the uploaded filename.

12. Maintenance Notes

When changes are made to the module, the following areas should be checked:

Change	Affected areas
New supported file extension	<code>uploadHandler()</code>
New VEC root elements	<code>validateVecFile()</code>
New KBL root elements	<code>validateKBLFile()</code>
Changed IODD structure rules	<code>validateIoddCore()</code> and IODD helper functions
Adjustment of error messages	Calling components and tests
Change in parser logic	<code>validateWellFormedXML()</code>
Parallel validation	Module-level state <code>file</code> , <code>errorMessage</code> , <code>resultMessage</code>

13. Known Risks

Risk	Impact	Recommendation
Module-level state	Parallel validations could interfere with one another	Prefer local state or serialize calls
Early exit	The user only sees the first error	Implement error accumulation if required
DOMParser behavior	Browser-dependent parser details may occur	Run tests in the target environment
Unknown KBL/VEC root variants	Valid files could be rejected	Review the root allowlist regularly
Other file extensions are not validated	Potentially unchecked files may proceed	Document this behavior deliberately or make validation stricter

IODD validation only checks core structure	Technically incomplete IODD files may be accepted	Add full IODD schema validation if needed
--	---	---

14. Outlook

In the future, the validation logic could be extended to support additional XML-based file formats as well as entirely different file types. This would make the module more flexible and allow it to cover a broader range of upload scenarios.